

Full Provenance on the COSS Causal Axis: An Event-Level Closure of Causal Attribution and Confused-Deputy Exclusion

Nicola Gallo

2 October 2026

Abstract

This paper follows the Provenance Identity Continuity (PIC) model and its relational extension beyond proof of possession. The earlier formulation represents an authority-use event as $(o, r, \ell) \in O \times R \times L$: an operation/resource behavior situated in a causal lineage. We make the causal occurrence itself primitive. Let $\mathcal{E}$ be the underlying occurrence universe and let $(E, \rightarrow_c)$, with $E \subseteq \mathcal{E}$, be the represented security-relevant causal graph. Each occurrence has a unique causal history $\text{Hist}(e)$, so the consistent event-lineage space $\hat{E} = \{(e, \ell) \in E \times L \mid \ell = \text{Hist}(e)\}$ is canonically isomorphic to E . The modeled execution has one causal partial order, not one clock or one order per lineage. Distinct lineages are distinct causal histories inside that order and may share the same root while diverging into incomparable branches. PIC’s $O \times R \times L$ is then the authority-behavior projection of this causal occurrence space. COSS generalizes the same construction by replacing the authority behavior $O \times R$ with an arbitrary behavior product $\mathcal{B}$ while retaining that one causal execution.

On this event semantics we prove an architecture-independent confused-deputy exclusion theorem. Ordinary PIC continuation is required on every ordinary represented causal edge: $C(e') \subseteq C(e)$. Consequently every descendant connected to an ancestor by an ordinary authority segment is bounded by that ancestor. At an ordinary multi-source join this is equivalent to $C(e) \subseteq \bigcap_{p \in \text{Pred}(e)} C(p)$; authority available on only one incoming branch therefore cannot be silently resurrected at the join, even when all branches share a common root. Common ancestry is weaker than continuation identity. Any legitimate non-conservative combination must be an explicit authorized composition or relaxation event. With protected-effect mediation, an accepted effect caused by request occurrence q and connected to it by an ordinary authority segment can exercise only authority in $C(q)$. Thus a confused-deputy state, which requires the same request-caused effect to exercise authority absent from $C(q)$ but independently held by the deputy, is contradictory. The result is universal over causally complete event models subject to the ordinary-segment premise: for any system whose protected request-to-effect causes are faith-

fully represented in the event DAG and whose ordinary authority transport satisfies PIC, no PIC lineage-form confused-deputy effect is admissible through ordinary continuation.

We then revisit Rajani, Garg, and Rezk (RGR). Their capability semantics blocks explicit designation but not all value, control-flow, or initial-heap attacks; their full provenance semantics tracks explicit and implicit influence and proves CDA-freedom with endorsement for their whole region calculus. From the causal PIC view, these attack classes are different witnesses of causal ancestry. Full provenance is therefore a concrete evidence mechanism for recovering data/control edges of the common causal graph, not the foundation of the PIC impossibility theorem. The paper finally lifts the construction to the COSS product of heterogeneous security-state coordinates. “Total” here is model-relative and ordinary-segment-relative: it covers all represented PIC lineage-form CDA instances whose request/effect pair satisfies the ordinary continuation premise, not physical channels or non-conservative transitions outside that scope.

1 Introduction

This work continues the PIC sequence: the original Proof-of-Continuity model binds authority to causal execution lineage, while *Beyond Proof of Possession* separates possession from admissible authority and makes explicit the role of request-caused effects [17, 18]. The present paper asks what changes when the execution occurrence itself, rather than an operation/resource tuple, is taken as the primitive object.

The answer is a causal generalization. A machine run consists of occurrences ordered by causal dependence. Operation, resource, principal, value flow, control decision, geographic location, software provenance, and other security facts are observations of those occurrences. They need not define separate times. PIC’s event (o, r, ℓ) is therefore read as the authority projection of a richer causal occurrence. COSS keeps the same causal axis and permits a behavior $b \in \mathcal{B}$ and a product security state to be attached to the same occurrence.

This viewpoint changes the confused-deputy problem.

The relevant structure is not the wall-clock distance between a request and a privileged effect. It is the branch-sensitive causal structure that reaches the effect. A request may influence an effect through a direct call, a value, a branch, mutable state, a queue, a database, a timer, a callback, a message, or an arbitrarily long chain. An authority-bearing branch may share an old root with the request yet still be a different causal continuation after a fork; it may also be concurrent with the request rather than ordered before it. In the event DAG these histories remain distinct until an explicit join. PIC then supplies the authority invariant: ordinary continuation cannot silently enlarge, restore, or union the authority carried from any causal parent merely because the parents have a common ancestor or meet in one process.

This observation yields the paper’s main theorem. If an accepted effect e is a causal descendant of a request q , and the segment from q to e is ordinary PIC continuation, then $C(e) \subseteq C(q)$. Since an accepted effect realizing (o, r) must have $(o, r) \in C(e)$, it follows that $(o, r) \in C(q)$. The lineage-form confused deputy requires the opposite. Hence no such state is admissible. The theorem is independent of whether the intermediate cause was called explicit designation, implicit designation, value influence, control flow, heap influence, or asynchronous state transfer.

The word *universal* is used here in a precise structural sense. The theorem quantifies over any system that admits a faithful, causally complete representation of its protected execution and enforces PIC on ordinary causal continuation. It is not a claim that a formal model automatically observes an unmodeled physical or covert cause. Complete causal representation and effect mediation are realization obligations.

After establishing this result, we revisit RGR’s 2016 formal analysis of capabilities and confused deputies [9]. Their counterexamples are especially useful because they exhibit several ways in which possession can be insufficient: the deputy may already possess the privileged capability while the adversary controls a value, a control decision, or state used later. We show that all of these can be represented as causal paths in the event model; whether such a path is an ordinary PIC authority segment is a separate premise. RGR’s full provenance then has a precise role: it can witness some of the data/control dependencies that populate the causal graph, while the PIC theorem itself is stated over the graph independently of that particular witness mechanism.

Contributions. The paper makes the following claims at the formal level.

1. It replaces the privilege tuple as the primitive event by a causal occurrence $e \in \mathcal{E}$, and gives the consistent representation $\hat{E} \subseteq E \times L$ from which PIC’s $O \times R \times L$ is obtained by authority-behavior projection.
2. It generalizes the event behavior from $O \times R$ to $\mathcal{B}$, giving the event-level basis of COSS without introducing

a second temporal axis.

3. It extends ordinary PIC authority continuity to causal DAGs, proving ancestor-bounded authority, common-root branch separation, and the conservative ordinary-join rule $C(e) \subseteq \bigcap_{p \in \text{Pred}(e)} C(p)$.
4. It defines causal/model completeness for protected confused-deputy reasoning and proves a universal schema: every causally represented, mediated PIC lineage-form confused-deputy effect on an ordinary segment is incompatible with ordinary PIC continuity.
5. It shows that temporal delay, common-root fan-out, and multi-source concurrency do not form separate CDA ontologies: they are path length, branch identity, and DAG-join structure in the same causal event model.
6. It rereads RGR’s explicit designation, value attack, implicit control influence, and initial-heap attack as causal-path instances, while preserving their original FPs theorem and terminology.
7. It gives a conservative event refinement of RGR FPs in which explicit and implicit provenance edges can serve as witnesses for causal edges, without attributing PoR, PoC, or event-lineage machinery to RGR.
8. It retains the COSS product construction, Product Continuity, projection/factorization results, heterogeneous translations, and explicit authorized relaxations as the general multi-coordinate extension of PIC.

2 From PIC Privilege Events to Causal Occurrences

2.1 Lamport-style causal event structure

Let $\mathcal{E}$ be the set of execution occurrences in a modeled run and let $E \subseteq \mathcal{E}$ be the occurrences retained by the claimed security model. The security-relevant causal graph is

$$\rightarrow_c \subseteq E \times E, \quad (1)$$

an acyclic represented-causality relation. An edge may correspond to an immediate machine-level cause or may collapse intervening occurrences omitted from E , but it must be causally sound for the modeled execution. Write $\prec_c = \rightarrow_c^+$ for its transitive closure. This is a happened-before-style *strict causal partial order*: no global physical clock or total order is assumed [15]. Sequential execution is a special case in which relevant events form a chain; concurrent or distributed execution is generally a DAG.

For $e \in E$, define its causal history

$$\text{Hist}(e) = \downarrow e := \{x \in E \mid x = e \vee x \prec_c e\}. \quad (2)$$

Let $L = \{\text{Hist}(e) \mid e \in E\}$ be the set of represented causal histories. In the original linear PIC model, a

represented history is a chain. In the DAG generalization used here, $\text{Hist}(e)$ may itself contain a fork and, after a join, several incomparable ancestors. Thus L denotes causal histories, not independent clocks and not merely process-local traces.

A consistent causal cut is a set $K \subseteq E$ that is downward closed:

$$e \in K \wedge x \prec_c e \implies x \in K. \quad (3)$$

A cut is the correct mathematical analogue of a coherent execution snapshot. It may contain tips of several incomparable branches; no wall-clock ordering between those tips is implied. Any total serialization chosen for analysis and required to respect $\prec_c$ is a linear extension of the causal partial order; such a serialization observes the execution without adding causal ancestry. The word *global* below therefore means that $\prec_c$ ranges over the whole modeled execution, not that the model assumes a globally synchronized physical clock.

The consistent event-lineage space is

$$\widehat{E} := \{(e, \ell) \in E \times L \mid \ell = \text{Hist}(e)\}. \quad (4)$$

Because Hist is a function of the occurrence, the map $e \mapsto (e, \text{Hist}(e))$ is a bijection from E to $\widehat{E}$. Thus $E \times L$ is useful as an explicit occurrence/history representation, but the history is not an independent clock.

PoR is the continuation witness used by PIC. At the semantic level,

$$\text{PoR}(e, e') \implies e \rightarrow_c e'. \quad (5)$$

A concrete system may realize this witness with signatures, attestations, runtime state, messages, provenance evidence, or another sound mechanism. The relation $\rightarrow_c$ is therefore the causal edge relation of the represented security graph; all state maps used below are defined on its vertices E .

2.2 PIC as an authority projection of the causal event space

Let

$$\text{beh}_{\text{auth}} : E \rightarrow O \times R \quad (6)$$

map an authority-relevant effect occurrence to the operation/resource behavior it realizes. The original PIC authority event

$$(o, r, \ell) \in O \times R \times L \quad (7)$$

is recovered from $e \in E$ whenever $\text{beh}_{\text{auth}}(e) = (o, r)$ and $\ell = \text{Hist}(e)$. Formally, PIC's event space is the image of the partial projection

$$\begin{aligned} \Psi_{\text{auth}} : \widehat{E} &\rightarrow O \times R \times L, \\ \Psi_{\text{auth}}(e, \text{Hist}(e)) &= (\text{beh}_{\text{auth}}(e), \text{Hist}(e)). \end{aligned} \quad (8)$$

Hence $O \times R \times L$ is not a second execution ontology. It is the behavior/history view needed by the authority model.

Each occurrence also carries an authority context

$$C : E \rightarrow \mathcal{P}(O \times R). \quad (9)$$

Definition 1 (Ordinary event and ordinary edge). *A non-origin occurrence $e \in E$ is ordinary when all of its security-relevant incoming causes are handled as non-expansive PIC continuations. If any incoming cause participates in an authorized grant, composition, or other non-conservative authority change, then e is classified as a non-ordinary composition/relaxation event. We write*

$$\text{Ord}(p, e) \iff p \rightarrow_c e \wedge e \text{ is ordinary}. \quad (10)$$

Thus ordinariness is resolved at the child occurrence, not independently per incoming edge; mixed ordinary/non-ordinary joins are classified as non-ordinary events.

For an ordinary causal edge, PIC continuity is

$$\text{Ord}(e, e') \implies C(e') \subseteq C(e). \quad (11)$$

This is exactly the original non-expansion rule, now stated directly on causal occurrences.

2.3 From authority behavior to general behavior

COSS changes the behavior projection, not the causal time. Fix a finite family of behavior spaces $(\mathcal{B}_j)_{j \in J}$ and define the typed product

$$\mathcal{B} := \prod_{j \in J} \mathcal{B}_j. \quad (12)$$

If only some facet combinations are meaningful, let $\mathcal{B}^* \subseteq \mathcal{B}$ denote that admissible behavior universe; otherwise take $\mathcal{B}^* = \mathcal{B}$. Let

$$\text{beh} : E \rightarrow \mathcal{B}^* \quad (13)$$

record the behavior realized by an occurrence. The corresponding explicit representation is

$$\widehat{E}_{\mathcal{B}} = \{(\text{beh}(e), \text{Hist}(e)) \mid e \in E\} \subseteq \mathcal{B}^* \times L. \quad (14)$$

With $\mathcal{B} = O \times R$ this recovers PIC. With $\mathcal{B} = \prod_j \mathcal{B}_j$, the same occurrence can expose authority, flow, provenance, identity transition, geography, or other behavior facets. What occurred and when/why it occurred are therefore separated cleanly: $\text{beh}(e)$ is the behavior, while $\text{Hist}(e)$ is its causal history.

2.4 Paths, common roots, concurrency, and joins

A lineage *path* is totally ordered by ancestry, but the full execution need not be. The first distinction needed for a DAG is between a common root and a common continuation. Let

$$r \prec_c a, \quad r \prec_c b, \quad a \not\prec_c b, \quad b \not\prec_c a. \quad (15)$$

Then a and b share the root r but lie on distinct causal branches.

Proposition 1 (Common ancestry does not identify causal histories). *Under Equation (15),*

$$\text{Hist}(a) \neq \text{Hist}(b), \quad (16)$$

even though $r \in \text{Hist}(a) \cap \text{Hist}(b)$.

Proof. By reflexivity of the history definition, $a \in \text{Hist}(a)$. Since a and b are incomparable, $a \notin \text{Hist}(b)$. Hence the histories differ. The common root belongs to both because $r \prec_c a$ and $r \prec_c b$.

Thus

$$\boxed{\text{same root} \not\Rightarrow \text{same causal continuation.}} \quad (17)$$

Co-residence in one process or one address space is weaker still: physical availability of two objects to one executor does not merge the causal histories by which those objects arrived.

Suppose now that two branch tips are both immediate causes of a later event e :

$$a \rightarrow_c e, \quad b \rightarrow_c e. \quad (18)$$

The event e is a causal join. The two incoming branches may have a common root as above, or they may have unrelated roots; the continuity question is the same at the join.

This matters because authority can attenuate differently after a fork. For example, let two privileges α and β be available at r , while ordinary continuations specialize the branches so that

$$C(a) = \{\alpha\}, \quad C(b) = \{\beta\}. \quad (19)$$

Both branches are valid descendants of the same root. Their later meeting does not, by itself, authorize the union $\{\alpha, \beta\}$. The security question is whether an ordinary join may recreate authority that one of its causal parents no longer carries. Section 3 answers no.

Operationally, $C(e)$ is a *causal authority envelope*: the effects that the continuation at occurrence e is permitted to realize. It is not the union of capabilities, credentials, references, or other objects physically reachable in the executor's memory. This distinction lets PIC reason about a process that simultaneously stores objects from several causal branches without treating co-residence as proof that their authority may be combined.

3 Causal PIC and Universal Confused-Deputy Exclusion

3.1 Ordinary joins cannot silently union authority

For $e \in E$, let $\text{Pred}(e) = \{p \in E \mid p \rightarrow_c e\}$ be its immediate causal predecessors in the security-relevant graph.

Definition 2 (Ordinary causal join). *A multi-parent occurrence e is an ordinary causal join when e is an ordinary event in the sense of Definition 1. Equivalently,*

$$\forall p \in \text{Pred}(e), \quad \text{Ord}(p, e). \quad (20)$$

An occurrence with any non-ordinary incoming security-relevant cause is classified as an explicit composition/relaxation event; Equation (11) is then not imposed on any of its incoming edges merely by calling another parent "ordinary."

Proposition 2 (Conservative ordinary-join authority). *If e is an ordinary causal join, then*

$$C(e) \subseteq \bigcap_{p \in \text{Pred}(e)} C(p). \quad (21)$$

Proof. For every $p \in \text{Pred}(e)$, Equation (11) gives $C(e) \subseteq C(p)$. Membership in every parent set is exactly membership in their intersection.

The proposition is deliberately conservative. It does not claim that every legitimate multi-party composition must use set intersection. It states what counts as *ordinary continuation*. If a system intentionally combines grants so that the child may exercise authority absent from one parent, that event is security-relevant precisely because it changes the continuity bound; it must therefore be represented as an explicit authorized composition/relaxation rather than hidden inside ordinary continuation. The rule applies equally when the parents have unrelated origins and when they are branches of the same earlier root.

For the common-root specialization of Equation (19), an ordinary join satisfies

$$C(e) \subseteq C(a) \cap C(b) = \emptyset. \quad (22)$$

The empty intersection is not a claim that either branch is invalid. It says that no authority remains valid as an *ordinary consequence of both branches simultaneously*. Any desired recombination must be represented as a separate authorized composition event.

Corollary 1 (No authority resurrection at an ordinary join). *Let e be an ordinary causal join and let (o, r) be a privilege. If there is any $p \in \text{Pred}(e)$ for which $(o, r) \notin C(p)$, then*

$$(o, r) \notin C(e). \quad (23)$$

Thus common ancestry cannot restore a privilege that is absent from one of the causal branches participating in the join.

Proof. By Proposition 2, $C(e) \subseteq C(p)$ for every $p \in \text{Pred}(e)$. The claim follows immediately.

Definition 3 (Ordinary authority segment on a DAG). *For $q, e \in E$ with $q \prec_c e$, write $\text{OrdSeg}(q, e)$ when there exists a causal path*

$$q = e_0 \rightarrow_c e_1 \rightarrow_c \cdots \rightarrow_c e_n = e \quad (24)$$

such that $\text{Ord}(e_{i-1}, e_i)$ for every $1 \leq i \leq n$. Because ordinariness is resolved at the child occurrence, a path cannot pass through a mixed or non-conservative join and still satisfy OrdSeg .

Theorem 1 (Ancestor-bounded authority on a causal DAG). *Let $q, e \in E$ with $q \prec_c e$. If $\text{OrdSeg}(q, e)$, then*

$$C(e) \subseteq C(q). \quad (25)$$

If every causal ancestor a of e also satisfies $\text{OrdSeg}(a, e)$, the same bound holds simultaneously for every such ancestor.

Proof. Choose a path $q = e_0 \rightarrow_c e_1 \rightarrow_c \dots \rightarrow_c e_n = e$. Equation (11) gives $C(e_n) \subseteq C(e_{n-1}) \subseteq \dots \subseteq C(e_0)$. Transitivity of set inclusion yields $C(e) \subseteq C(q)$. The simultaneous statement follows by applying the same argument to each ancestor path; at a multi-parent ordinary join, Proposition 2 preserves all parent bounds.

3.2 Causal completeness and effect mediation

Let $\text{Req} \subseteq E$ be request occurrences and $\text{Eff} \subseteq E$ protected effect occurrences, and let $\mathcal{D}$ be the set of executor instances. Use the typed relations and maps

$$\text{CausedBy} \subseteq \text{Eff} \times \text{Req}, \quad (26)$$

$$\text{Accepted} \subseteq \text{Eff}, \quad (27)$$

$$\text{realize} : \text{Eff} \rightarrow O \times R, \quad (28)$$

$$\text{exec} : \text{Eff} \rightarrow \mathcal{D}, \quad (29)$$

$$\text{IndependentAuthority} \subseteq \mathcal{D} \times (O \times R). \quad (30)$$

Here CausedBy is the semantic request/effect causation relation for the stated threat model, $\text{Accepted}(e)$ abbreviates $e \in \text{Accepted}$, realize maps an effect to the operation and actual resource it realizes, and $\text{IndependentAuthority}(D, p)$ states that executor D independently possesses privilege $p \in O \times R$.

Definition 4 (CDA-complete causal event representation). *A modeled execution is CDA-complete for a stated threat model when:*

(C1) *every protected effect in scope is represented by an event in Eff ;*

(C2) *whenever $\text{CausedBy}(e, q)$ holds for an in-scope request q and protected effect e , the represented causal graph preserves that semantic causation as ancestry:*

$$\text{CausedBy}(e, q) \implies q \prec_c e. \quad (31)$$

The implication in (C2) is intentionally one-way. Causal ancestry can exist without the earlier occurrence being a semantic cause of the particular effect under study; the theory does not identify all happened-before pairs with request/effect causation.

When a claim additionally reasons about the provenance of independently held authority, we use the following optional strengthening.

Definition 5 (Authority-source completeness). *An execution is authority-source complete for a privilege (o, r) at an effect e when every in-scope independent authority source that contributes that privilege is represented by an occurrence or initial causal state in the ancestry relevant to e .*

Condition (C2) is a representation-completeness condition, not an additional security policy. Full provenance, trusted runtime tracing, message causality, explicit read-from edges, or another mechanism may supply evidence that the represented ancestry is faithful; Section 5 examines one such mechanism.

Protected-effect mediation is a separate enforcement premise:

$$\begin{aligned} e \in \text{Eff} \wedge \text{Accepted}(e) \wedge \text{realize}(e) = (o, r) \\ \implies (o, r) \in C(e). \end{aligned} \quad (32)$$

Proposition 3 (Represented causal attribution). *In a CDA-complete causal event representation,*

$$\text{CausedBy}(e, q) \implies q \prec_c e \quad (33)$$

and therefore $q \in \text{Hist}(e)$.

Proof. The ancestry implication is condition (C2). By the definition of $\text{Hist}(e)$ in Equation (2), $q \prec_c e$ implies $q \in \text{Hist}(e)$.

The proposition records the model/semantics interface rather than deriving causality from an unrelated label. A concrete implementation must justify that the represented graph is faithful enough for condition (C2).

3.3 Request-bounded effects

Lemma 1 (Request-origin authority bound). *If $\text{CausedBy}(e, q)$ and $\text{OrdSeg}(q, e)$, then*

$$C(e) \subseteq C(q). \quad (34)$$

Proof. By Proposition 3, $q \prec_c e$. The premise $\text{OrdSeg}(q, e)$ then lets us apply Theorem 1.

Theorem 2 (Request-bounded effect theorem). *Let q be a request occurrence and e an accepted protected effect. Then*

$$\begin{aligned} \text{CausedBy}(e, q) \wedge \text{OrdSeg}(q, e) \wedge \text{Accepted}(e) \\ \wedge \text{realize}(e) = (o, r) \implies (o, r) \in C(q). \end{aligned} \quad (35)$$

Proof. Lemma 1 gives $C(e) \subseteq C(q)$. Effect admission (32) gives $(o, r) \in C(e)$. Hence $(o, r) \in C(q)$.

3.4 Lineage-form confused deputy and branch-divergent witnesses

Definition 6 (Causal lineage-form confused deputy). *For $q \in Req$, $e \in Eff$, $D \in \mathcal{D}$, and $p \in O \times R$, define*

$$\begin{aligned} & CDA_{PIC}(q, e, D, p) \iff \\ & \text{CausedBy}(e, q) \wedge \text{Accepted}(e) \\ & \wedge \text{realize}(e) = p \wedge \text{exec}(e) = D \\ & \wedge p \notin C(q) \wedge \text{IndependentAuthority}(D, p). \end{aligned} \quad (36)$$

This is the lineage-form authority-mismatch property excluded below on ordinary request-to-effect segments; it is not identified with RGR’s extensional CDAF-E definition.

When the model also records the causal provenance of the deputy’s independent authority, let

$$\text{AuthSource} \subseteq E \times \mathcal{D} \times (O \times R) \quad (37)$$

and write $\text{AuthSource}(a, D, (o, r))$ when occurrence a contributes D ’s independent authority for (o, r) . If an implementation models pre-execution state as a distinguished initial source, that source is included in E . If $\text{AuthSource}(a, D, (o, r))$ and $a \prec_c e$, then the same CDA instance has an explicit branch/source witness. The source a may share an older root with q and still belong to a distinct continuation after a fork. This structural witness is useful for join analysis but is not required for the contradiction in Theorem 3.

Theorem 3 (Universal causal-PIC exclusion on ordinary segments). *For every system whose protected execution has a faithful CDA-complete causal event representation, whose ordinary causal edges satisfy PIC authority continuity, and whose protected effects satisfy Equation (32),*

$$\begin{aligned} & \forall q \in Req, \forall e \in Eff, \forall D \in \mathcal{D}, \forall p \in O \times R : \\ & CDA_{PIC}(q, e, D, p) \wedge \text{OrdSeg}(q, e) \implies \perp. \end{aligned} \quad (38)$$

Proof. Fix arbitrary q, e, D, p satisfying the antecedent. By $CDA_{PIC}(q, e, D, p)$ we have $\text{CausedBy}(e, q)$, $\text{Accepted}(e)$, and $\text{realize}(e) = p$. Together with $\text{OrdSeg}(q, e)$, Theorem 2 yields $p \in C(q)$. The same CDA predicate requires $p \notin C(q)$. Hence $p \in C(q) \wedge p \notin C(q)$, a contradiction. Independent authority held by D cannot enlarge the request-carried authority through ordinary continuation; at a multi-source join Proposition 2 prevents silent union. A legitimate non-conservative combination must cross an explicit authorized composition or relaxation boundary and is therefore outside the ordinary-segment antecedent.

Corollary 2 (Temporal distance does not create a new CDA class). *Suppose $q \prec_c e$ and an ordinary request-to-effect path witnessing $\text{OrdSeg}(q, e)$ contains arbitrarily many intermediate events or arbitrarily long wall-clock delay. The conclusion of Theorem 2 is unchanged.*

Proof. The proof depends only on causal ancestry and transitivity of set inclusion, not on path length or physical time elapsed.

Corollary 3 (Temporal proximity does not create causal ancestry). *Let $e_1, e_2 \in E$ be incomparable under $\prec_c$:*

$$e_1 \not\prec_c e_2 \quad \wedge \quad e_2 \not\prec_c e_1. \quad (39)$$

Wall-clock proximity, simultaneous observation, co-membership in a consistent cut, or placement next to one another in a total serialization does not create a causal relation between them. In particular, such observations do not merge their causal histories or their authority justifications.

Proof. The relation $\prec_c$ is the transitive closure of the modeled immediate-causality relation $\rightarrow_c$. A consistent cut records events without adding edges, and a total serialization that respects $\prec_c$ is only a linear extension of the same partial order. Neither operation changes $\rightarrow_c$ or its transitive closure. Hence incomparable events remain incomparable, and their histories remain distinct unless a later modeled causal event joins them.

The two temporal corollaries are dual: causal ancestry survives arbitrary physical delay, while physical proximity does not manufacture ancestry.

The main implication is therefore

causally complete protected event model + ordinary PIC authority continuity + protected-effect mediation + ordinary request-to-effect segment $\implies$ every represented PIC lineage-form CDA effect is inadmissible along ordinary continuation.	(40)
--	------

This is a universal theorem schema over represented systems, not a claim that an omitted physical cause has somehow been modeled.

4 Rajani–Garg–Rezk: What Capabilities and Full Provenance Prove

Rajani, Garg, and Rezk (RGR) study access control, capabilities, provenance tracking, and confused-deputy attacks in a small imperative region calculus [9]. Their result is important here because it gives a formal taxonomy of attacks that possession-style capability checks do and do not prevent.

Their capability semantics implements Miller et al.’s Property A, “no designation without authority,” by preventing a region from obtaining a write capability it is not authorized to wield. RGR show, however, that this does

not imply CDA-freedom in general. The capability semantics blocks the classical *explicit designation* case in which the attacker provides the privileged capability/reference itself, but it does not block all cases in which the deputy already possesses the privileged capability and the attacker influences how it is used.

Their Examples 2–4 distinguish three such shapes. A *value attack* lets the adversary control a value that the privileged deputy later writes to a high-integrity reference. Their Example 3, *Implicit influence*, is a control-flow attack in which an adversary-controlled condition determines which privileged write occurs. An *initial-heap attack* lets a privileged capability already present in state be used later under adversarial influence. RGR call these forms of *implicit designation*. Their Theorem 4 obtains CDA-freedom with endorsement for capability semantics only under the stated *nihH* and *nihP* restrictions on the initial heap and non-endorsed high regions.

RGR next add provenance tracking. Their explicit-only semantics (EPs) tracks principals whose references explicitly influence a computed value, but omits implicit control-flow influence. Their full provenance semantics (FPs) adds a program-counter label stack: on entry to an *if* or *while* body, the current *pc* is combined with the condition label, and the assignment rule checks target provenance, value provenance, and the current *pc* against the owner of the written reference. In this way FPs enforces both explicit and implicit integrity dependencies. Their Theorem 6 states

$$\text{CDAF-E}(E_{\rho_A}, H, \rightarrow_{\text{FP}}) \text{ holds unconditionally} \quad (41)$$

for their region calculus and FPs semantics. Their proof says that FPs enforces a strong form of information-flow integrity (their Definition 8), which in turn implies CDA-freedom with endorsement through their Theorem 7. Theorem 7 states that NI-A-E implies CDAF-E for each of their four semantics, for every initial heap; they also state that the converse does not hold [9]. “Unconditionally” is relative to their formal calculus: it removes the additional heap/program restrictions needed by their capability and EP results.

RGR do not define PoR, PoC, a distributed lineage protocol, or a Lamport event DAG. Their labels are principal-valued abstractions of influence. The causal interpretation below is therefore ours, and it is used only to relate their attack classes and provenance mechanism to the event semantics already defined.

4.1 RGR attack classes as causal paths

Under a causally complete embedding, the RGR examples differ in the concrete witness of ancestry, not in the PIC

theorem that applies afterward:

$$\text{explicit: } q \rightarrow_c \text{ capability transfer } \rightarrow_c e, \quad (42)$$

$$\text{value: } q \rightarrow_c \text{ data/state dependency } \rightarrow_c e, \quad (43)$$

$$\text{control: } q \rightarrow_c \text{ branch dependency } \rightarrow_c e, \quad (44)$$

$$\text{initial heap: } q \rightarrow_c \text{ read-from dependency } \rightarrow_c e. \quad (45)$$

The paths may contain arbitrarily many additional events. What matters is $q \prec_c e$.

Proposition 4 (RGR-class closure under causal PIC). *Consider any execution of the four RGR attack shapes embedded in a CDA-complete causal PIC model so that the relevant capability-transfer, value, control, or state dependency is represented in $\prec_c$. If $\text{OrdSeg}(q, e)$, then an accepted privileged effect $\text{realize}(e) = (o, r)$ requires $(o, r) \in C(q)$. Therefore an RGR-style attack in which the request origin lacks (o, r) is inadmissible.*

Proof. Each embedding supplies $q \prec_c e$ and the proposition assumes $\text{OrdSeg}(q, e)$. Theorem 2 then gives the claim independently of which dependency shape witnessed the ancestry.

This proposition does not identify RGR’s extensional CDAF-E definition with the PIC lineage-form definition, nor does it claim that Theorem 3 is a proof of RGR’s Theorem 6. It shows something narrower and structural: once an attack instance is represented as request-to-effect causal ancestry, PIC excludes the authority mismatch by the same argument for all four shapes.

The branch view also clarifies a subtle LOW/HIGH case. Saying that an attacker-controlled LOW value and a HIGH authority object are both “in the same execution” or even descend from the same old root does not decide the security question. If they reach an effect through different causal branches, the join must preserve the authority bound of every branch participating in that effect. Conversely, if the HIGH privilege is already valid in the causal authority envelope of every relevant parent, authority-PIC alone need not reject the effect merely because LOW influenced its value or control flow. RGR’s non-interference property can still reject such an influence. This is why

$$\text{NI-A-E} \implies \text{CDAF-E} \quad (46)$$

does not collapse IFC into PIC: non-interference is a stronger information-flow property, while PIC constrains causal transport of authority. COSS can carry both as distinct coordinates on the same causal event structure.

5 Full Provenance as Evidence on the COSS Causal Axis

The universal PIC theorem is stated over the causal event structure and does not require RGR full provenance. FPs is nevertheless useful as a concrete language-level way to expose some of the edges that a causally complete model must represent, especially explicit data influence and implicit control influence.

5.1 Conservative event refinement of FPs

Consider an RGR FPs execution

$$\Theta_0 \rightarrow_{\text{FP}} \Theta_1 \rightarrow_{\text{FP}} \cdots \rightarrow_{\text{FP}} \Theta_n. \quad (47)$$

Associate a ghost occurrence $e_i \in E$ with each reduction step. Define an influence relation $\rightsquigarrow_{\text{FP}}$ by two generators:

- (F1) if a value, reference, target designation, or heap value produced or obtained at e_i is a semantic input to the computation at e_j , then $e_i \rightsquigarrow_{\text{FP}} e_j$;
- (F2) if execution of e_j is control-dependent on a branch or loop condition evaluated at e_i , then $e_i \rightsquigarrow_{\text{FP}} e_j$.

The transitive closure is $\rightsquigarrow_{\text{FP}}^+$.

Proposition 5 (Conservative erasure). *Erasing the ghost event identifiers and $\rightsquigarrow_{\text{FP}}$ edges yields exactly the underlying RGR FPs trace, with the same principal-valued labels, PC evolution, and accepted or blocked assignments.*

Proof. The event identifiers and dependency edges occur in no premise of an RGR FPs reduction rule. Erasure therefore commutes with each reduction step; induction on the trace length gives the result.

Lemma 2 (Full-provenance influence is causally ordered). *If the causal event semantics represents producer/consumer and condition/control dependencies faithfully, then*

$$e_i \rightsquigarrow_{\text{FP}} e_j \implies e_i \prec_c e_j, \quad \rightsquigarrow_{\text{FP}}^+ \subseteq \prec_c. \quad (48)$$

Proof. An explicit FPs dependency means the later step consumes a value/reference or state whose relevant production/read occurred earlier; semantic dependency therefore places the producer/read in the causal past of the consumer. An implicit FPs dependency means the later command is reachable only through a branch/loop condition whose label contributes to the active *pc*; the condition occurrence is therefore in the causal past of the controlled occurrence. Transitivity gives the closure statement.

The reverse inclusion need not hold. Causal ordering may exist without data or control influence. Thus FPs can witness an influence suborder of the common execution; it does not define a second time and is not required to make PIC's authority theorem true.

5.2 Evidence role in asynchronous and persistent systems

The same semantic idea can be realized by other witnesses. If a later event consumes state or a message produced by an earlier event, a causally complete system can expose

dependencies such as

$$\text{write}_x \rightarrow_c \text{read}_x, \quad (49)$$

$$\text{enqueue}_q \rightarrow_c \text{dequeue}_q, \quad (50)$$

$$\text{send}_m \rightarrow_c \text{receive}_m, \quad (51)$$

$$\text{setTimer} \rightarrow_c \text{fireTimer}, \quad (52)$$

$$\text{registerCallback} \rightarrow_c \text{invokeCallback}. \quad (53)$$

A database, queue, timer, callback, shared object, or message broker therefore does not constitute a new category of confused deputy merely because the original synchronous stack has disappeared. The mathematical requirement is that the cause remain represented in the ancestry of the effect.

6 Historical Motivation: Different Security Coordinates

Bell and LaPadula developed a mathematical model of secure computer states and rules intended to preserve security under state transitions [1, 2]. Denning subsequently formulated secure information flow using a lattice of security classes, giving a general mathematical view of systems that restrict information flow [3].

Capability systems start from a different primitive. Authority is represented by protected designators that permit operations on objects. In 1984, Boebert showed that an *unmodified* capability machine, in the absence of additional mechanisms, cannot enforce the Bell–LaPadula *-property in his model [4]. His concrete attack lets a low-clearance program place a read/write capability for a low object into that low object; a later high-clearance program may read the low object, obtain the capability, and then combine read access to high data with write access to the low object. Boebert's concluding explanation is that, in the pure capability machine he defines, the right to exercise access carries the ability to propagate that access.

Miller, Yee, and Shapiro revisited this construction directly in *Capability Myths Demolished* [5]. They agree that the particular construction described by Boebert fails to enforce the *-property, but argue that the attack assumes capabilities can be transmitted wherever data can be transmitted. In partitioned or type-enforced capability systems, capability transfer and data transfer are distinguishable operations. Their two-level construction independently controls those operations and is presented as a counterexample to the broader claim that capability systems as a class cannot enforce the *-property. The same paper also stresses that a static access matrix is not a complete description of a security mechanism: the rules by which access relationships evolve over time materially affect the security properties of the resulting system.

Miller and Shapiro develop a complementary argument in *Paradigm Regained* [6]. They distinguish permission from authority and explicitly make program behavior

on permitted causal pathways part of authority analysis. Their object-capability model carries overt causation by messages over permitted paths. Their examples use Redell’s Caretaker pattern for revocation, factories for overt confinement, and a data diode for the $*$ -properties. They also show how multiple independently motivated abstractions can coexist over the same graph under “mutually suspicious composition.”

These papers therefore already contain dynamic, behavioral, and causal reasoning; COSS does not claim otherwise. The distinction made here is representational. In those analyses, causal pathways and abstraction behavior are derived from the reference graph and program semantics. COSS instead makes causal lineage an explicit axis of the security event and transports an ordered security state along that axis. Nor does COSS assume that a capability implementation can contribute only an authority coordinate: a capability design may itself mediate information flow, transfer type, request context, or another security distinction. The general claim is narrower: if an enforcement decision is invariant under a distinction that changes the target security verdict, that decision cannot enforce the target property. A typed transfer rule or access abstraction may expose exactly the distinction that a coarser model omitted.

This paper does *not* claim that Boebert, Bell–LaPadula, Denning, Miller, or Shapiro formulated their work in this product language. The product view is a new interpretation intended to explain why apparently conflicting results can all be correct under different observable state spaces.

7 Ordered Security-State Systems

Definition 7 (Security-state coordinate). *A security-state coordinate is a tuple*

$$\mathfrak{S}_j = (X_j, \preceq_j, \mathcal{B}_j, \llbracket \cdot \rrbracket_j), \quad (54)$$

where:

- X_j is a set of security states;
- $\preceq_j$ is a partial order interpreted as “no more permissive than”;
- $\mathcal{B}_j$ is a universe of behaviors, transitions, principals, flows, or other semantically admitted possibilities; and
- $\llbracket x \rrbracket_j \subseteq \mathcal{B}_j$ is the semantic extension admitted by state $x \in X_j$.

The coordinate is semantically monotone when

$$x' \preceq_j x \implies \llbracket x' \rrbracket_j \subseteq \llbracket x \rrbracket_j. \quad (55)$$

The definition deliberately does not require every security system to be born as a powerset. A lattice label, a logical predicate, a capability set, a provenance constraint, or a history-dependent state can all be used if they admit a sound refinement order.

Remark 1 (Coordinates are semantic dimensions). *A coordinate is not required to correspond one-to-one with an implementation mechanism. One mechanism may realize several security coordinates, as when a type-enforced capability runtime distinguishes authority from data transfer and capability transfer; conversely, several mechanisms may jointly realize one coordinate. COSS reasons over the security distinctions exposed to the admission decision, not over product names or implementation categories.*

Remark 2 (Refinement is not a complete update semantics). *The order $\preceq_j$ is the condition needed for the continuity theorem, not a claim that every refinement step is reachable or permitted by the concrete security system. A coordinate may impose a stricter local transition relation $U_j \subseteq X_j \times X_j$ satisfying*

$$U_j(x, x') \implies x' \preceq_j x. \quad (56)$$

All ordinary-continuation results below remain valid when transitions must satisfy both U_j and the refinement order. This leaves room for the richer update rules that distinguish dynamic security mechanisms even when they share the same static state representation.

Definition 8 (Product security state). *For a finite family J of security-state coordinates, define*

$$X = \prod_{j \in J} X_j, \quad \Sigma = (\sigma^{(j)})_{j \in J}. \quad (57)$$

The product refinement order is

$$\Sigma' \preceq \Sigma \iff \forall j \in J, \sigma'^{(j)} \preceq_j \sigma^{(j)}. \quad (58)$$

For a concrete execution, let

$$\text{st} : E \rightarrow X \quad (59)$$

assign exactly one product security state to each execution occurrence, and write $\Sigma_i := \text{st}(s_i)$. If $\ell \in L$ is a finite causal history with endpoint s_n , define

$$\Sigma(\ell) := \text{st}(s_n). \quad (60)$$

Thus the state used by an event is a well-defined function of the endpoint occurrence of its represented history.

Proposition 6 (Product order). *If every $(X_j, \preceq_j)$ is a partial order, then $(X, \preceq)$ defined by (58) is a partial order.*

Proof. Reflexivity, antisymmetry, and transitivity hold componentwise.

Remark 3 (Coordinates need not be independent). *The Cartesian product is an ambient state space, not an assumption that every combination of coordinate states is meaningful. If cross-coordinate constraints permit only states in $Y \subseteq X$, the induced order on Y is still a partial order and Product Continuity applies unchanged to lineages whose states remain in Y . Likewise, if only certain combinations of behavior facets are meaningful, one may restrict the event universe to $\mathcal{B}^* \subseteq \prod_j \mathcal{B}_j$ and interpret product admission on $\mathcal{B}^* \cap \prod_j \llbracket \sigma^{(j)} \rrbracket_j$.*

8 Causally Ordered Security-State Continuity

Definition 9 (Valid COSS transition). *A transition*

$$(s_i, \Sigma_i) \longrightarrow (s_{i+1}, \Sigma_{i+1}) \quad (61)$$

is COSS-valid when

$$\text{PoR}(s_i, s_{i+1}) \quad \wedge \quad \Sigma_{i+1} \preceq \Sigma_i. \quad (62)$$

Equivalently, every security coordinate refines or preserves its predecessor state while the execution advances along a verified causal edge.

Definition 10 (Product security-state continuity). *A lineage*

$$\Pi = \langle (s_0, \Sigma_0), \dots, (s_n, \Sigma_n) \rangle \quad (63)$$

has product security-state continuity when every adjacent transition is COSS-valid.

Theorem 4 (Product Continuity). *If Π has product security-state continuity, then*

$$\Sigma_n \preceq \Sigma_{n-1} \preceq \dots \preceq \Sigma_0. \quad (64)$$

Hence, for every coordinate j ,

$$\sigma_n^{(j)} \preceq_j \sigma_0^{(j)}. \quad (65)$$

If the coordinate is semantically monotone, then

$$\llbracket \sigma_n^{(j)} \rrbracket_j \subseteq \llbracket \sigma_0^{(j)} \rrbracket_j. \quad (66)$$

Proof. Every valid transition gives $\Sigma_{i+1} \preceq \Sigma_i$. Transitivity of the product order gives $\Sigma_n \preceq \Sigma_0$. Projection to coordinate j gives (65); semantic inclusion follows from (55).

The theorem states the general continuity invariant for lineages containing only ordinary continuations: causal transport cannot introduce a possibility forbidden by the origin state of any carried security system.

Theorem 5 (Pathwise Product Continuity on a causal DAG). *Let $q, e \in E$ with $q \prec_c e$. If every edge of a causal path from q to e is an ordinary COSS transition, then*

$$\text{st}(e) \preceq \text{st}(q). \quad (67)$$

If every immediate predecessor edge of a multi-parent ordinary join is required to satisfy the same componentwise non-expansion rule, the bound holds along every parent path.

Proof. Choose a causal path $q = s_0 \rightarrow_c s_1 \rightarrow_c \dots \rightarrow_c s_k = e$. Each ordinary edge gives $\text{st}(s_{i+1}) \preceq \text{st}(s_i)$. Transitivity yields $\text{st}(e) \preceq \text{st}(q)$. The multi-parent statement follows because the same argument applies independently to every immediate-predecessor path.

Definition 11 (Product admission). *The product state $\Sigma = (\sigma^{(j)})_{j \in J}$ admits $b \in \mathcal{B}^*$ iff $b^{(j)} \in \llbracket \sigma^{(j)} \rrbracket_j$ for every j . Write*

$$\llbracket \Sigma \rrbracket := \mathcal{B}^* \cap \prod_{j \in J} \llbracket \sigma^{(j)} \rrbracket_j \subseteq \mathcal{B}^*. \quad (68)$$

Proposition 7 (Joint semantic monotonicity). *If every coordinate is semantically monotone, then $\Sigma' \preceq \Sigma$ implies $\llbracket \Sigma' \rrbracket \subseteq \llbracket \Sigma \rrbracket$. Consequently, under product security-state continuity, $\llbracket \Sigma_n \rrbracket \subseteq \llbracket \Sigma_0 \rrbracket$.*

Proof. $\Sigma' \preceq \Sigma$ gives $\sigma'^{(j)} \preceq_j \sigma^{(j)}$ for every j , hence $\llbracket \sigma'^{(j)} \rrbracket_j \subseteq \llbracket \sigma^{(j)} \rrbracket_j$ by semantic monotonicity, and intersection with the fixed set $\mathcal{B}^*$ preserves inclusion. The second claim follows from Product Continuity.

A later occurrence therefore admits only behaviors that the origin state admits in every coordinate jointly, not merely coordinate by coordinate.

9 Why Refinement Can Be Read as Set Inclusion

The previous theorem is intentionally stated over partial orders rather than subsets. Nevertheless, the subset intuition can be recovered for every partial order.

Definition 12 (Principal lower-set representation). *For a poset $(X, \preceq)$ define*

$$\downarrow x = \{y \in X \mid y \preceq x\}. \quad (69)$$

Theorem 6 (Principal down-set embedding). *For every poset $(X, \preceq)$, the standard principal down-set map*

$$\eta : X \rightarrow \mathcal{P}(X), \quad \eta(x) = \downarrow x \quad (70)$$

is an order embedding:

$$x' \preceq x \iff \downarrow x' \subseteq \downarrow x. \quad (71)$$

Proof. If $x' \preceq x$ and $y \preceq x'$, transitivity gives $y \preceq x$, so $\downarrow x' \subseteq \downarrow x$. Conversely, $\downarrow x' \subseteq \downarrow x$ implies $x' \in \downarrow x$ because $x' \in \downarrow x'$ by reflexivity. Hence $x' \preceq x$.

This standard principal down-set embedding formalizes the subset intuition behind the present work. A security system need not literally store a set, but its refinement order can be represented faithfully as inclusion among sets of states no more permissive than the current one. This inclusion must not be confused with semantic admission: $\downarrow x \subseteq X$ is a set of *states*, whereas $\llbracket x \rrbracket \subseteq \mathcal{B}$ is a set of *behaviors*. Semantic monotonicity connects refinement to behavior inclusion, but the two constructions are distinct. The familiar PIC rule $C_{i+1} \subseteq C_i$ is therefore one concrete instance of a more general ordered refinement pattern.

10 Projection, Quotients, and Coordinate Blindness

The second PIC paper interprets lineage-forgetting as a quotient and shows that lineage-invariant decisions factor through the projection that forgets causal history [18]. The same construction applies to arbitrary products of security-state coordinates.

Let

$$X = \prod_{j \in J} X_j \quad (72)$$

and let $K \subseteq J$. Define the coordinate projection

$$p_K : X \rightarrow X_K := \prod_{j \in K} X_j. \quad (73)$$

Definition 13 (*K*-invariant decision). *A decision rule $A : X \rightarrow \{0, 1\}$ is K -invariant when*

$$p_K(x) = p_K(y) \implies A(x) = A(y). \quad (74)$$

Theorem 7 (Projection Factorization). *A decision rule $A : X \rightarrow \{0, 1\}$ is K -invariant if and only if there exists a unique rule*

$$\bar{A} : p_K(X) \rightarrow \{0, 1\} \quad (75)$$

such that

$$A = \bar{A} \circ p_K. \quad (76)$$

Proof. If $A = \bar{A} \circ p_K$, equal projections yield equal decisions. Conversely, define $\bar{A}(z) = A(x)$ for any x with $p_K(x) = z$. K -invariance makes the definition independent of the representative. Uniqueness follows because p_K is surjective onto $p_K(X)$.

Corollary 4 (Omitted-coordinate impossibility). *Suppose a target security property $P : X \rightarrow \{0, 1\}$ distinguishes two states $x, y \in X$ for which $p_K(x) = p_K(y)$. No K -invariant decision rule can equal P on all of X .*

Proof. Every K -invariant rule must give the same result on x and y , while P requires different results.

This is the general form of the earlier lineage-forgetting observation: a security decision cannot enforce a distinction that its observation function erases.

Corollary 5 (Behavioral-state blindness). *Let $X = X_{\text{prot}} \times X_{\text{beh}}$ separate a primitive protection-state coordinate from a coordinate describing security-relevant behavior or an access abstraction. If a target property $P : X \rightarrow \{0, 1\}$ satisfies*

$$P(p, h_1) \neq P(p, h_2) \quad (77)$$

for some fixed $p \in X_{\text{prot}}$, then no decision rule that factors through the projection $(p, h) \mapsto p$ can implement P on all of X .

Proof. This is Omitted-coordinate Impossibility with $K = \{\text{prot}\}$.

This corollary gives an order-independent statement of the blind spot discussed by Miller and Shapiro: if two systems have the same protection graph but the behavior of a security-enforcing abstraction changes the authority or flow verdict, an analysis that observes only the protection graph is necessarily too coarse for that distinction [6]. It does not imply that protection-state analysis is unsound when it succeeds; it states only that such a projection cannot reproduce a behavior-sensitive property in general.

Using the endpoint-state assignment of (60), the COSS admission rule for an event (b, ℓ) is

$$A_{\text{COSS}}(b, \ell) = 1 \iff b \in \llbracket \Sigma(\ell) \rrbracket. \quad (78)$$

Let $q_L : \mathcal{B}^* \times L \rightarrow \mathcal{B}^*$, $(b, \ell) \mapsto b$, forget the lineage.

Corollary 6 (Lineage-forgetting blindness in COSS). *Let $b \in \llbracket \Sigma(\ell) \rrbracket \setminus \llbracket \Sigma(\ell') \rrbracket$. No decision rule that factors through q_L equals A_{COSS} on both (b, ℓ) and (b, ℓ') .*

Proof. $q_L(b, \ell) = q_L(b, \ell')$, so every rule factoring through q_L gives both events the same verdict, while $A_{\text{COSS}}(b, \ell) = 1$ and $A_{\text{COSS}}(b, \ell') = 0$. This is the same fiber argument as Projection Factorization, now applied to the projection that omits L from the event space.

For the authority coordinate alone this is the lineage-invariance result of PIC [17]. In COSS it holds for every carried security system at once: the same behavior can be admissible in one causal history and inadmissible in another, in authority, in information flow, in relational admissibility or in any other coordinate, and a decision that forgets the history cannot tell the two apart.

10.1 A three-coordinate example

Consider a three-step agent lineage

$$s_0 \rightarrow s_1 \rightarrow s_2 \quad (79)$$

carrying authority, information-flow, and geographic coordinates

$$\Sigma_i = (C_i, F_i, G_i), \quad \Sigma_2 \preceq \Sigma_1 \preceq \Sigma_0. \quad (80)$$

At s_2 , suppose C_2 still permits the operation (send, reportAPI) and G_2 admits the agent's current EU execution region, but F_2 excludes the flow facet f_{ext} representing transmission of the current data to an external sink. Let b be the event facet tuple whose authority component is that send operation, whose flow component is f_{ext} , and whose geographic component is the current EU region. Authority and geography admit their respective facets, but IFC does not, so

$$b \notin \llbracket \Sigma_2 \rrbracket \quad (81)$$

and the product rejects the event. An authority-only decision would admit the event even though the complete

product state rejects it: the authority coordinate itself remains valid, while the IFC coordinate does not.

Now consider a different causal history ℓ' carrying the same authority and geographic states but an IFC state F' that admits f_{ext} , for example after a separately authorized declassification. If ℓ denotes the first history ending at s_2 , the same event facet tuple b may satisfy

$$b \in \llbracket \Sigma(\ell') \rrbracket \quad \text{while} \quad b \notin \llbracket \Sigma(\ell) \rrbracket. \quad (82)$$

The example exhibits both claims of the theory: all relevant security coordinates constrain one causal occurrence jointly, and a decision that forgets the lineage cannot distinguish histories whose current admissibility differs.

11 Classical Security Models as Coordinates

11.1 Authority and capabilities

Let O be operations and R resources. Define

$$X_{\text{auth}} = \mathcal{P}(O \times R), \quad C' \preceq_{\text{auth}} C \iff C' \subseteq C. \quad (83)$$

This coordinate says which authority may participate in the current causal execution. It does not identify the executor that happens to possess other ambient authority.

With this coordinate alone, COSS-validity reduces to

$$\text{PoR}(s_i, s_{i+1}) \quad \wedge \quad C_{i+1} \subseteq C_i, \quad (84)$$

which is precisely the abstract continuity rule of PIC [17].

This specialization is compatible with the stronger object-capability results in the literature. *Capability Myths Demolished* identifies absence of ambient authority, inseparability of designation and authority, and access-controlled delegation channels as important properties of object-capability systems; it also emphasizes that authority conveyed in the context of a request helps avoid the classical confused-deputy pattern [5]. COSS does not treat those protections as missing. A capability system may realize several security distinctions at once; the lineage coordinate matters only for a target property whose verdict also varies with causal execution after the capability-level distinctions have been fixed.

11.2 Information-flow admissibility

Let D be a domain of information sources and sinks, security classes, or other flow endpoints. One possible semantic IFC coordinate is the set of currently admitted flows

$$F \subseteq D \times D. \quad (85)$$

Define

$$F' \preceq_{\text{ifc}} F \iff F' \subseteq F. \quad (86)$$

A Bell–LaPadula or lattice policy can induce such an admissible-flow relation; this paper does not replace the

internal semantics of those models. It treats their effective admissibility state as an independent coordinate whose ordinary continuation must not become more permissive.

An alternative is to take lattice labels or complete IFC policy states as X_{ifc} and define $\preceq_{\text{ifc}}$ directly from their semantic flow sets. Equation (86) is therefore an example, not a claim that all IFC implementations literally store F .

The policy state F_i should be distinguished from the dynamic provenance of a particular run. The former answers which flows are admissible; the latter answers which earlier occurrences actually influenced a value or control decision in this execution. RGR’s FPs supplies a formal influence abstraction through value labels and the program-counter stack. Section 5 lifts that influence to event resolution without changing the original FPs accept/reject behavior.

11.3 Typed data and capability transfer

The distinction used by Miller, Yee, and Shapiro to revisit Boebert can itself be represented as an ordered security coordinate without claiming that their systems used this notation. Let Λ be a set of security domains and let

$$K = \{\text{data}, \text{cap}\}, \quad \mathcal{B}_{\text{tr}} = \Lambda \times \Lambda \times K \quad (87)$$

classify a transfer by source domain, destination domain, and whether the transferred value is ordinary data or a capability. Define

$$X_{\text{tr}} = \mathcal{P}(\mathcal{B}_{\text{tr}}), \quad (88)$$

$$Q' \preceq_{\text{tr}} Q \iff Q' \subseteq Q, \quad \llbracket Q \rrbracket_{\text{tr}} = Q. \quad (89)$$

The coordinate can therefore admit a data transfer between two levels while rejecting a capability transfer over the same pair of levels.

This captures the specific distinction made in *Capability Myths Demolished*: Boebert’s attack requires a high-clearance subject to obtain, by reading a low object, a capability that confers write access to that low object; the later type-enforced construction distinguishes capability movement from data movement and blocks the cross-level capability transfer [5]. The coordinate Q is only an illustrative semantic encoding of that distinction, not a claim that an object-capability implementation literally stores a set of typed flows.

11.4 Relational admissibility

The relational extension of PIC carries a predicate ρ_i over successor/predecessor profiles and orders predicates by implication [18]. After quotienting by semantic equivalence, define

$$X_{\text{rel}} = \text{Pred}(\mathcal{T}_J) / \equiv_{\text{sem}}, \quad (90)$$

with

$$[\rho'] \preceq_{\text{rel}} [\rho] \iff \rho' \Rightarrow \rho. \quad (91)$$

Semantically,

$$\rho' \Rightarrow \rho \iff \llbracket \rho' \rrbracket \subseteq \llbracket \rho \rrbracket. \quad (92)$$

Thus the existing PIC product

$$\Gamma_i = (C_i, [\rho_i]_{\text{sem}}) \quad (93)$$

is already a two-coordinate instance of COSS.

11.5 Geographic, provenance, and product constraints

A geographic restriction can be represented by an admissible region set G_i^{allow} ordered by inclusion, while the observed current location g_i is checked for membership. A software-product or provenance coordinate can analogously represent the set of admissible artifact identities, builders, attestations, or derivation histories. These coordinates do not need to share an ontology with authority or IFC; they need only expose a sound refinement order.

This distinction is useful for invocation. A product can participate in many runtime invocations, just as one executor can realize many execution occurrences. Product identity is therefore not itself an invocation. A future DAG extension can treat invocation as a causal join between an authority lineage and a product/provenance lineage while preserving the coordinates that justify each parent.

12 Re-reading the Capability/IFC Tension

Consider the two-coordinate state

$$\Sigma = (C, F) \in X_{\text{auth}} \times X_{\text{ifc}}. \quad (94)$$

A decision that reads only C factors through the projection

$$p_C(C, F) = C. \quad (95)$$

If two states (C, F_1) and (C, F_2) require different information-flow verdicts, any C -invariant mechanism must nevertheless treat them identically. This is not a statement that capability systems are intrinsically incapable of information-flow enforcement. It is a statement about observational insufficiency: the decision cannot enforce a distinction represented only in a coordinate it does not observe.

This gives a compact reinterpretation of the historical sequence without turning it into a contest between security paradigms. Boebert's oracle does observe clearance and classification when it grants a capability; the failure in his construction occurs in subsequent propagation. Even an oracle that correctly creates capabilities cannot, in the unmodified machine he defines, control their further propagation [4]. The distinction here is therefore between the grant decision and what later executions may acquire and exercise, not between a labeled and an unlabeled grant.

The later capability responses act directly on that propagation assumption. Miller and Shapiro's data diode transmits data in one direction and capabilities in neither [6]; Miller, Yee, and Shapiro separately show how distinguishing capability transfer from data transfer blocks the cross-level capability movement required by Boebert's attack, while remaining within their reading of his definition [5]. Under a COSS encoding, an oracle grant that enlarges the carried authority would be an origination or explicitly authorized relaxation, whereas acquisition of a new capability during ordinary continuation cannot silently enlarge the authority coordinate. This is a COSS interpretation of the security distinction, not a claim that Boebert or the later capability papers used this state model.

Miller and Shapiro further show that program behavior and access abstractions can tighten authority beyond what an arrangement-only analysis of the protection graph predicts [6]. In COSS terms, these constructions are examples of mechanisms that make additional distinctions observable or restrict the semantic state on which the decision operates. Once those distinctions are observed, the mechanism no longer belongs to the projection-blind class covered by the impossibility corollary. The blindness claim concerns the information read when an event is admitted; it does not say that the original capability-granting oracle ignored clearance or classification.

The same reasoning generalizes. If

$$\Sigma = (S_1, S_2, \dots, S_m), \quad (96)$$

a policy that factors only through S_1 cannot enforce a property whose verdict varies with S_2 while S_1 is fixed. Security models can therefore be individually valid and still be jointly necessary.

13 PIC as the Authority Specialization of COSS

The relationship between COSS and PIC is a generalization by security-state dimension, while the causal-PIC confused-deputy theorem of Section 3 remains the authority specialization.

PIC, first form. Take a single coordinate

$$X = X_{\text{auth}} = \mathcal{P}(O \times R). \quad (97)$$

Then COSS-validity is exactly PoR plus authority attenuation, and Product Continuity becomes PIC's origin-binding result.

Relational PIC. Take

$$X = X_{\text{auth}} \times X_{\text{rel}}. \quad (98)$$

Then the product order is

$$(C', \rho') \preceq (C, \rho) \iff C' \subseteq C \wedge \rho' \Rightarrow \rho, \quad (99)$$

which is the product attenuation order already used by the relational theory [18].

General form. The natural closure is

$$X = \prod_{j \in J} X_j, \quad (100)$$

where authority, IFC, relational admissibility, provenance, geography, separation-of-duty memory, or another security system may contribute a coordinate. PIC is therefore not renamed or invalidated. It remains the authority-continuity specialization from which the more general product construction was discovered.

14 COSS in the PIC Series

COSS is the third step in a sequence of models that share the same basic shape: authorization is attached to an execution occurrence in causal history, PoR relates adjacent occurrences, a carried state is ordered by refinement, and ordinary continuation may preserve or restrict that state but not silently expand it. At the level of this continuity construction, each later paper contains the earlier state discipline as a specialization; this statement does not imply that the later construction reproduces every implementation theorem or deployment assumption of the earlier papers.

The containment statements are exact at the level claimed here. With $J = \{\text{auth}\}$, the COSS transition rule is PoR plus $C_{i+1} \subseteq C_i$, the validity rule of the first PoC model. With $J = \{\text{auth}, \text{rel}\}$ and relational refinement ordered by implication, the product order is the relational PIC attenuation order $(C', \rho') \preceq (C, \rho)$ iff $C' \subseteq C$ and $\rho' \Rightarrow \rho$ [18].

The separation results share the same factorization principle but should not be identified more strongly than their domains permit. For the authority-only event space, take $b = (o, r)$ and two histories ℓ, ℓ' such that $(o, r) \in C(\ell)$ and $(o, r) \notin C(\ell')$. The COSS lineage-forgetting corollary then gives the same required separation as the first paper's lineage-invariance result. The relational paper subsequently shows that the earlier lineage-forgetting projection is one quotient instance of a more general factorization construction; its executor-profile quotients and COSS's coordinate projections are applications of the same mathematical principle, not literally the same projection map [18].

The confused-deputy result is strengthened in the present paper. Hardy's classical example motivates the problem of a deputy exercising authority for the wrong request context [16]. The first PoC paper excludes silent authority import inside a valid lineage through origin-bounded attenuation, and the relational paper separates possession from admissible request authority [18]. Section 2 makes the occurrence itself primitive; Section 3 then lifts PIC to causal DAGs and proves that every represented request-caused protected effect is bounded by the

request origin on ordinary continuation. RGR full provenance is used later as one evidence mechanism for causal edges, not as a premise of the authority impossibility theorem.

15 Heterogeneous Security-State Spaces

PIC already permits policy translation between heterogeneous operation/resource spaces [17]. The same idea extends componentwise.

At hop i , let coordinate j use state space $X_i^{(j)}$ with order $\preceq_i^{(j)}$. Let

$$T_{i \rightarrow i+1}^{(j)} : X_i^{(j)} \rightarrow X_{i+1}^{(j)} \quad (101)$$

be an order-theoretic policy translation. A translated transition satisfies

$$\sigma_{i+1}^{(j)} \preceq_{i+1}^{(j)} T_{i \rightarrow i+1}^{(j)}(\sigma_i^{(j)}). \quad (102)$$

If the translations are monotone and composable, define

$$T_{0 \rightarrow n}^{(j)} = T_{n-1 \rightarrow n}^{(j)} \circ \dots \circ T_{0 \rightarrow 1}^{(j)}. \quad (103)$$

Proposition 8 (Translated continuity). *If every $T_{i \rightarrow i+1}^{(j)}$ is monotone and every transition satisfies (102), then*

$$\sigma_n^{(j)} \preceq_n^{(j)} T_{0 \rightarrow n}^{(j)}(\sigma_0^{(j)}). \quad (104)$$

Proof. By induction on n . For $n = 0$ the claim is reflexivity. Suppose $\sigma_i^{(j)} \preceq_i^{(j)} T_{0 \rightarrow i}^{(j)}(\sigma_0^{(j)})$. Monotonicity of $T_{i \rightarrow i+1}^{(j)}$ gives $T_{i \rightarrow i+1}^{(j)}(\sigma_i^{(j)}) \preceq_{i+1}^{(j)} T_{0 \rightarrow i+1}^{(j)}(\sigma_0^{(j)})$, and (102) with transitivity gives the claim for $i + 1$.

Proposition 8 is order-theoretic: it requires monotonicity of the translations and proves a bound in the target orders. A stronger claim that a translation preserves the intended security meaning across heterogeneous behavior universes requires a separate semantic relation between those universes. This paper does not define such a cross-domain semantic soundness condition; an overly permissive semantic translation is therefore a policy or deployment error rather than a failure of the order-theoretic continuity theorem.

16 Ordinary Continuation versus Authorized Relaxation

Not every legitimate security transition is monotone. De-classification may relax an IFC constraint; a new authority grant may enlarge an authority state; a policy administrator may broaden an admissible geographic region.

COSS therefore distinguishes two classes of transition.

Definition 14 (Ordinary continuation). *An ordinary continuation is a PoR-linked transition satisfying $\Sigma_{i+1} \preceq \Sigma_i$ (or the translated form of (102)).*

	PoC [17]	Relational PIC [18]	COSS
Event domain	$O \times R \times L$	execution occurrences indexed by causal histories	$\widehat{E}_B \subseteq \mathcal{B}^* \times L$
Carried state	$C \subseteq O \times R$	$(C, [\rho]_{\text{sem}})$	$\Sigma \in \prod_j X_j$
Order	$\subseteq$	$\subseteq$ and $\Rightarrow$, componentwise	$\preceq_j$, componentwise
Continuity result	origin-bounded authority	Dual Monotonicity and Combined Closure	Product Continuity
Separation result	lineage-invariant policies cannot individuate occurrences	quotient factorization and observation-based indistinguishability	Projection Factorization and lineage-forgetting blindness

Table 1: The PIC series as a layered continuity construction.

Definition 15 (Authorized relaxation event). *An authorized relaxation event is an explicitly distinguished causal event whose validity is justified by a separate authority or policy rule and which may replace one or more coordinates by states not below their predecessors.*

The point is not to prohibit declassification, re-authorization, or policy change. It is to prevent those events from being confused with ordinary continuation. A security state may become more permissive only at a boundary whose authority to perform that relaxation is itself explicit and auditable.

This mirrors the distinction already present in PIC between attenuation within one lineage and the creation of new authority at an origin or explicit future composition boundary.

Relaxation events are still causal events: they remain causally linked to their represented predecessor(s), while the ordinary monotonicity obligation is replaced by the separate authorization/composition rule.

Proposition 9 (Segment continuity). *Let a PoR-linked lineage contain authorized relaxation events at indices $1 \leq r_1 < \dots < r_k$, where the event at r_m is the transition from s_{r_m-1} to s_{r_m} , all other transitions being ordinary continuations. For every i , let $r(i)$ be the largest $r_m \leq i$, or 0 if none exists. Then*

$$\Sigma_i \preceq \Sigma_{r(i)}. \quad (105)$$

The statement assumes a homogeneous state space along the segment. For segments containing translated hops, the corresponding bound is the one given by Proposition 8.

Proof. If $r(i) = i$, the claim is reflexivity. Otherwise, every transition leaving $s_{r(i)}$ and ending at or before s_i is an ordinary continuation, because $r(i)$ is the most recent relaxation-entry index. Product Continuity applied to the segment from $s_{r(i)}$ through s_i gives $\Sigma_i \preceq \Sigma_{r(i)}$.

Origin-boundedness is therefore replaced, in the presence of relaxation, by boundedness from the most recent explicitly authorized relaxation. A relaxation cannot be introduced by ordinary continuation; it can only restart the bound at an explicit, auditable point.

17 Causal DAGs, Joins, and Multiple Causes

The causal event model is a DAG rather than a requirement of one linear history. There is one partial order for the modeled execution, while different lineages are different histories or paths inside that order. In particular, a common root does not collapse later branches into one continuation (Proposition 1).

The authority coordinate admits a sufficient conservative discipline for ordinary joins: Proposition 2 requires

$$C(e) \subseteq \bigcap_{p \in \text{Pred}(e)} C(p). \quad (106)$$

This is sufficient for the universal confused-deputy theorem because it prevents authority available from only one incoming branch from being silently relabelled as authority of the joined effect. Corollary 1 adds the stronger common-root reading: authority lost on one branch cannot be restored merely because another branch retained it or because both branches share an ancestor.

For the full COSS product, however, not every coordinate must compose by set intersection. A future general join operator

$$\kappa_j : X_j^k \rightarrow X_j^\kappa \quad (107)$$

must state the semantics of legitimate multi-parent composition for coordinate j and must preserve the provenance of the parent states. If the result is more permissive than one or more parents, the join is not ordinary continuation for that coordinate and must be justified as an explicit authorized composition/relaxation event.

Thus the present paper supplies a sufficient conservative DAG discipline for ordinary confused-deputy authority mixing, while leaving the algebra of non-conservative multi-coordinate composition as a broader COSS research problem. The distinction is important: no general product-join theorem is needed to prove that an *ordinary* authority join cannot silently union independent permissions.

18 Related Work

Bell–LaPadula. Bell and LaPadula model secure system states together with rules of operation that preserve security under state transitions [2]. COSS does not replace those rules or claim that Bell–LaPadula is merely a set-inclusion policy. Rather, an effective Bell–LaPadula security state may serve as one coordinate when its local refinement relation and admitted-event semantics are specified; COSS adds the explicit causal-event axis and the product transport rule used when that state must coexist with other security states.

Denning’s lattice model. Denning organizes security classes as a lattice whose order is justified by the semantics of permitted information flow [3]. This is a direct example of an ordered security domain, but the IFC semantics remain Denning’s: COSS supplies neither the lattice nor a program analysis for discovering flows. It supplies a way to carry such an ordered state together with other coordinates along one causal execution.

Execution-history access control. Abadi and Fournet make run-time rights depend on execution history rather than only the current stack [7]. In their model, ordinary execution automatically intersects current rights with the static rights of code that runs, while an explicit special operation may modify current rights, at most restoring the static rights of that code and subject to further constraints. This is structurally close to the distinction here between ordinary non-expansive continuation and an explicit relaxation event. The models nevertheless have different domains: their construction computes a rights state for code execution history, whereas COSS abstracts over an arbitrary finite product of security-state coordinates and takes sound PoR as an explicit premise relating the modeled execution occurrences.

Decentralized information-flow control. Myers and Liskov define decentralized labels with owner-specific reader sets, a restriction order that forms a security-class lattice, and explicit declassification that is legal only when the process has authority to act for the relevant owner [8]. Their restriction/declassification distinction is therefore an important prior instance of monotone security restriction plus explicitly authorized relaxation. COSS treats such an IFC state as one possible coordinate; it does not replace the label semantics, acts-for relation, or static flow analysis of that work.

Rajani–Garg–Rezk full provenance. RGR develop what they describe as the first formal characterization of CDA-freedom in a language-based setting, and later give a formal characterization of a language fragment in which capabilities can provably prevent CDAs. They also show that capability semantics does not prevent all implicit-designation CDAs and introduce explicit-only and full

provenance semantics [9]. FPs augments value provenance with a program-counter label stack so that branch and loop conditions contribute to the integrity check on later writes. Their Theorem 6 proves CDA-freedom with endorsement unconditionally for their region calculus, via information-flow integrity and their Theorem 7. Read from COSS, FPs is a language-level witness for explicit and implicit influence edges that may populate the causal structure of Section 2. COSS does not reinterpret their theorem as a protocol result. The new step is a conservative event refinement that places FPs-style influence on the same execution occurrences as PoR; the causal-PIC exclusion theorem itself is stated over the event ancestry independently of this particular witness.

Unified authorization and dependency calculi. Flow-limited authorization integrates authorization and information-flow reasoning for delegation and trust [10]; the Dependency Core Calculus gives one calculus in which several dependency analyses, including information flow, are instances [11]; and operating-system DIFC designs track labels and privileges across processes [12, 13, 14]. These works unify security reasoning within a language semantics or an operating-system label model. COSS starts from a different object: authority bound to execution occurrences in a causal history that may span services and trust domains, with continuity verifiable at the receiving boundary. The approaches are complementary: a language-based or operating-system analysis can supply the influence relation, or a security coordinate, that COSS transports along PoR.

Capability systems and access abstractions. Miller, Yee, and Shapiro show that conclusions about capability systems depend on the capability model and on distinctions such as data transfer versus capability transfer, designation with authority, and access-controlled delegation channels [5]. Miller and Shapiro further show that security-enforcing program behavior and access abstractions can restrict authority beyond what a protection-graph-only analysis exposes [6]. COSS’s claim is not that these systems lack context, flow control, or causal reasoning. Its contribution is a uniform event/state construction and a factorization result: whenever a target verdict depends on a distinction erased by an observation, a decision invariant under that observation cannot reproduce the target verdict.

19 Semantic Closure and Realization Assumptions

Theorem 3 is architecture-independent but not assumption-free. Its universality is over systems that faithfully realize the causal model. A concrete realization must justify four obligations.

1. *Causal completeness.* Every protected request-caused effect and every causal dependency relevant to the claimed threat model must be represented in the event DAG. This includes branch identity across protocol handoffs and, when in scope, dependencies mediated by memory, storage, queues, messages, timers, callbacks, or other state. A cause omitted from the model is not covered by the theorem.
2. *Sound causal evidence.* PoR, provenance, runtime tracing, read-from relations, message metadata, or other evidence used to assert an edge must be sound for the semantic causality being claimed.
3. *Complete protected-effect mediation.* Every protected effect in scope must be checked against the authority state of its occurrence, as required by Equation (32).
4. *Explicit non-conservative transitions.* Endorsement, declassification, authority grant, or multi-source composition that enlarges a coordinate must be separately authorized and represented as such rather than hidden inside ordinary continuation.

These obligations explain the relation between the present paper and the relational PIC paper. The earlier theory kept Causal Attribution and Complete Mediation as deployment assumptions. Here semantic request/effect causation remains distinct from happened-before, but CDA-completeness requires every in-scope causal-attribution fact to be preserved as event ancestry; Complete Mediation is the protected-effect admission rule. The proof is therefore closed inside the formal event model once those representation and enforcement obligations are satisfied; deployment correctness consists in showing that the implementation faithfully realizes them.

20 Limitations and Open Questions

Physical and covert channels. The theorem is total over represented PIC lineage-form CDA instances whose request/effect pair satisfies the ordinary-segment premise. A physical influence channel omitted from the causal event semantics is outside the quantification rather than a counterexample inside it. Claims about covert or side channels therefore require instrumentation or a semantics that actually places the relevant influence in the event graph.

Non-conservative product joins. The authority rule for ordinary joins is closed by Proposition 2. General COSS coordinates may require coordinate-specific join algebras for legitimate non-conservative composition. Those joins must remain explicit so that they cannot masquerade as ordinary continuation.

Semantic soundness of heterogeneous translations.

Translated Continuity proves preservation relative to supplied ordered translation maps. Relating states and behaviors across different vocabularies still requires an application-specific semantic relation and soundness condition.

Order soundness and coordinate choice. A mathematically monotone but semantically incorrect refinement order proves the wrong property. Likewise, omitting a coordinate is safe only for target properties invariant under that omission; the projection theorem makes this boundary explicit.

Implementation evidence. The theory is agnostic about whether causal and security evidence is carried in tokens, attestations, trusted-runtime state, gateways, proof-carrying objects, dynamic IFC tracking, or another mechanism. The obligation is semantic soundness, not a particular representation.

21 Discussion

The conceptual move of the paper is to treat the execution occurrence as the common object. PIC originally wrote an authority event as (o, r, ℓ) . The causal formulation makes $e \in \mathcal{E}$ primitive and obtains (o, r, ℓ) by projection. COSS then replaces the single authority behavior with a product behavior $b \in \mathcal{B}$ and carries heterogeneous ordered security state on the same occurrence.

This makes several distinctions precise. First, causal time is not the same as wall-clock time: arbitrary delay does not break ancestry, and a coherent snapshot is represented by a consistent cut rather than by assuming a global physical instant. Second, concurrency does not require two independent “times”: incomparable branches live in the same partial order and may meet at a join. Third, sharing a root does not make those branches the same lineage. If authority attenuates differently after a fork, the join cannot reconstruct the root union by ordinary continuation. Fourth, a branch-divergent confused deputy can be exposed as an authority-provenance problem at such a join; the core lineage-form theorem itself requires only request causation, an authority mismatch, and an ordinary segment. If one causal branch is a request whose context lacks (o, r) and another branch carries independent deputy authority for (o, r) , an ordinary PIC join cannot silently produce a child that exercises (o, r) because the child is bounded by every causal parent/ancestor. Intentional authority composition is possible, but it must be explicit and separately authorized.

RGR’s results fit this picture without being rewritten. Their capability counterexamples identify concrete ways in which the deputy can already possess a privileged reference while the adversary controls a value, branch, or stored state. Their FPs semantics provides an information-flow mechanism that tracks these influences inside their

language. Causal PIC abstracts one level lower: once the request-to-effect dependency is represented in the event DAG, the same ancestor-bound authority theorem applies regardless of the syntactic carrier of the influence.

The resulting division of labor is therefore:

Causal structure answers *which occurrence caused which later occurrence*; PIC answers *which authority may continue along that causal structure*; COSS generalizes the carried state to other security dimensions; and concrete provenance/attestation mechanisms provide evidence that the modeled causal and state relations hold in an implementation.

This is why the confused-deputy result is both strong and carefully scoped. It is not merely detection: a protected PIC lineage-form CDA instance on an ordinary request-to-effect segment is not an admissible ordinary PIC state. But the theorem is only as physically comprehensive as the causal model and mediation path that a deployment actually realizes.

22 Conclusion

This paper starts from the PIC and relational PIC models and makes their causal object explicit. A security-relevant machine execution is represented by one Lamport-style strict causal partial order $(E, \prec_c)$, with $E \subseteq \mathcal{E}$. Distinct lineages are distinct causal histories inside that order; they may share a root and still diverge into incomparable branches. The consistent pair $(e, \text{Hist}(e)) \in E \times L$ is the canonical occurrence/history object; PIC's $(o, r, \ell) \in O \times R \times L$ is its authority-behavior projection, and COSS's $(b, \ell) \in \mathcal{B}^* \times L$ is the corresponding general behavior projection. There is one causal execution, not separate PIC and IFC times. Here “one causal execution” means one causal partial order: distinct lineages are different histories within the same DAG, not positions on a shared global timeline.

On that structure ordinary PIC continuity is local and simple: $C(e') \subseteq C(e)$ on each ordinary causal edge. Its transitive consequence is pathwise: whenever $\text{OrdSeg}(q, e)$ holds, $C(e) \subseteq C(q)$. At a multi-source ordinary join,

$$C(e) \subseteq \bigcap_{p \in \text{Pred}(e)} C(p), \quad (108)$$

so authority that belongs only to one incoming branch cannot be silently imported into the joined effect. In particular,

$$\text{same root} \not\Rightarrow \text{same continuation}, \quad (109)$$

and common ancestry cannot resurrect authority attenuated away on a branch.

With protected-effect mediation, this yields

$$\begin{aligned} \text{CausedBy}(e, q) \wedge \text{OrdSeg}(q, e) \wedge \text{realize}(e) = (o, r) \\ \wedge \text{Accepted}(e) \implies (o, r) \in C(q). \end{aligned} \quad (110)$$

A lineage-form confused deputy requires the same effect to satisfy $(o, r) \notin C(q)$. Therefore the two conditions imply

$$(o, r) \in C(q) \wedge (o, r) \notin C(q), \quad (111)$$

and hence contradiction.

RGR's capability counterexamples do not create exceptions to this causal statement. Explicit designation, value influence, implicit control influence, and initial-heap influence are different concrete witnesses of request-to-effect ancestry. Their full provenance semantics is a powerful language-level mechanism for exposing explicit and implicit influence and independently proves CDA-freedom in their region calculus. In the present theory it can also serve as evidence for causal edges, but it is not required as the logical foundation of PIC authority continuity. Nor does authority-PIC subsume all of non-interference: if every relevant causal parent already authorizes a HIGH effect, a LOW-to-HIGH influence can remain an IFC violation without being an authority-expansion violation. The two properties are distinct coordinates on the same causal execution.

The final result is therefore:

Every represented PIC lineage-form CDA instance
on an ordinary request/effect segment is
incompatible with ordinary PIC continuity.

(112)

Equivalently, for every system satisfying causal completeness, protected-effect mediation, and ordinary PIC authority continuity,

$\forall q \in \text{Req}, \forall e \in \text{Eff}, \forall D \in \mathcal{D}, \forall p \in O \times R :$
 $\text{CDA}_{\text{PIC}}(q, e, D, p) \wedge \text{OrdSeg}(q, e) \implies \perp.$

(113)

The quantifier is universal over systems satisfying that representation and enforcement contract. The theorem does not assert coverage of physical causes omitted from the event model, nor does it exclude executions that cross an explicitly authorized non-conservative authority transition. It states that once the relevant causes and protected effects are inside the causal semantics, no PIC lineage-form confused-deputy instance is an admissible ordinary PIC continuation.

References

- [1] D. E. Bell and L. J. LaPadula, “Secure Computer Systems: Mathematical Foundations,” The MITRE Corporation, ESD-TR-73-278, Vol. I, 1973.
- [2] L. J. LaPadula and D. E. Bell, “Secure Computer Systems: A Mathematical Model,” The MITRE Corporation, ESD-TR-73-278, Vol. II, 1973.
- [3] D. E. Denning, “A Lattice Model of Secure Information Flow,” *Communications of the*

- ACM, vol. 19, no. 5, pp. 236–243, 1976, doi: 10.1145/360051.360056.
- [4] W. E. Boebert, “On the Inability of an Unmodified Capability Machine to Enforce the *-Property,” in *Proceedings of the 7th DoD/NBS Computer Security Conference*, Gaithersburg, MD, USA, Sep. 24–26, 1984, pp. 291–293. [Online]. Available: <https://csrc.nist.gov/files/pubs/conference/1984/09/24/7th-dodnbs-computer-security-conference/final/docs/1984-7th-conference-proceedings.pdf>.
 - [5] M. S. Miller, K.-P. Yee, and J. S. Shapiro, “Capability Myths Demolished,” Technical Report SRL2003-02, Systems Research Laboratory, Department of Computer Science, Johns Hopkins University, Mar. 2003.
 - [6] M. S. Miller and J. S. Shapiro, “Paradigm Regained: Abstraction Mechanisms for Access Control,” in *Advances in Computing Science – ASIAN 2003: Programming Languages and Distributed Computation*, Lecture Notes in Computer Science, vol. 2896, Springer, 2003, pp. 224–242, doi: 10.1007/978-3-540-40965-6_15.
 - [7] M. Abadi and C. Fournet, “Access Control Based on Execution History,” in *Proceedings of the 10th Annual Network and Distributed System Security Symposium (NDSS)*, 2003.
 - [8] A. C. Myers and B. Liskov, “A Decentralized Model for Information Flow Control,” in *Proceedings of the 16th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 129–142, 1997, doi: 10.1145/268998.266669.
 - [9] V. Rajani, D. Garg, and T. Rezk, “On Access Control, Capabilities, Their Equivalence, and Confused Deputy Attacks,” in *2016 IEEE 29th Computer Security Foundations Symposium (CSF)*, Lisbon, Portugal, 2016, doi: 10.1109/CSF.2016.18.
 - [10] O. Arden, J. Liu, and A. C. Myers, “Flow-Limited Authorization,” in *2015 IEEE 28th Computer Security Foundations Symposium (CSF)*, 2015.
 - [11] M. Abadi, A. Banerjee, N. Heintze, and J. G. Riecke, “A Core Calculus of Dependency,” in *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 1999.
 - [12] P. Efstathopoulos, M. Krohn, S. VanDeBogart, C. Frey, D. Ziegler, E. Kohler, D. Mazières, F. Kaashoek, and R. Morris, “Labels and Event Processes in the Asbestos Operating System,” in *Proceedings of the 20th ACM Symposium on Operating Systems Principles (SOSP)*, 2005.
 - [13] N. Zeldovich, S. Boyd-Wickizer, E. Kohler, and D. Mazières, “Making Information Flow Explicit in HiStar,” in *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2006.
 - [14] M. Krohn, A. Yip, M. Brodsky, N. Cliffer, M. F. Kaashoek, E. Kohler, and R. Morris, “Information Flow Control for Standard OS Abstractions,” in *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP)*, 2007.
 - [15] L. Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978, doi: 10.1145/359545.359563.
 - [16] N. Hardy, “The Confused Deputy (or Why Capabilities Might Have Been Invented),” *ACM SIGOPS Operating Systems Review*, vol. 22, no. 4, pp. 36–38, 1988.
 - [17] N. Gallo, “Proof-of-Continuity: A Temporal Model for Authority Propagation in Distributed Systems and AI Agents,” arXiv:2607.08906, 2026.
 - [18] N. Gallo, “Beyond Proof of Possession: A Relational Theory of Authority,” manuscript, 2026.